

Regresiones No Lineales y Polinómicas

Transformación de Características, Funciones de Costo y Métricas

Diego Stalder

August 31, 2026

Contenido de la Clase

- 1 El Problema de la Linealidad
- 2 Funciones de Costo para Regresión
- 3 Métricas de Evaluación
- 4 Fundamento Matemático
- 5 Implementación con Scikit-Learn
- 6 Ejemplo 1: Curva de Calibración de Sensor
- 7 Análisis de Residuos
- 8 Ejemplo 2: Rendimiento Académico
- 9 Regularización
- 10 Interpretación
- 11 Resumen
- 12 Referencias

¿Por qué necesitamos regresión no lineal?

- El modelo lineal tiene la forma:

$$y = w_0 + w_1x_1 + w_2x_2 + \cdots + w_nx_n$$

- **Limitación:** asume una relación lineal entre variables.
- En la vida real, muchas relaciones son **curvilíneas**:
 - Calibración de sensores (curvas de respuesta)
 - Crecimiento poblacional
 - Rendimiento académico
 - Conteo de piezas defectuosas

¿Por qué necesitamos regresión no lineal?

- El modelo lineal tiene la forma:

$$y = w_0 + w_1x_1 + w_2x_2 + \cdots + w_nx_n$$

- **Limitación:** asume una relación lineal entre variables.
- En la vida real, muchas relaciones son **curvilíneas**:
 - Calibración de sensores (curvas de respuesta)
 - Crecimiento poblacional
 - Rendimiento académico
 - Conteo de piezas defectuosas

Transformamos las características originales añadiendo términos polinómicos:

$$y = w_0 + w_1x + w_2x^2 + w_3x^3 + \cdots + w_dx^d$$

Función de Costo: Mean Squared Error (MSE)

$$\text{MSE}(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

donde $\hat{y}_i = \mathbf{w}^T \phi(\mathbf{x}_i)$

Función de Costo: Mean Squared Error (MSE)

$$\text{MSE}(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

donde $\hat{y}_i = \mathbf{w}^T \phi(\mathbf{x}_i)$

- **Convexa:** tiene un único mínimo global
- **Diferenciable:** podemos usar gradiente descendente
- **Sensibilidad a outliers:** errores grandes se amplifican al cuadrado

Función de Costo: Mean Squared Error (MSE)

$$\text{MSE}(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

donde $\hat{y}_i = \mathbf{w}^T \phi(\mathbf{x}_i)$

- **Convexa:** tiene un único mínimo global
- **Diferenciable:** podemos usar gradiente descendente
- **Sensibilidad a outliers:** errores grandes se amplifican al cuadrado

$$\mathbf{w} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

$$\text{MAE}(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$$

Propiedades: robusta a outliers.

$$\text{MAE}(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$$

Propiedades: robusta a outliers.

$$L_{\delta}(y, \hat{y}) = \begin{cases} \frac{1}{2}(y - \hat{y})^2 & \text{si } |y - \hat{y}| \leq \delta \\ \delta|y - \hat{y}| - \frac{1}{2}\delta^2 & \text{en otro caso} \end{cases}$$

- Combina MSE y MAE
- δ controla el umbral

Regresión Poisson (Datos de Conteo)

- Variable objetivo son conteos: $y_i \in \{0, 1, 2, 3, \dots\}$
- Distribución Poisson: $P(y = k) = \frac{\lambda^k e^{-\lambda}}{k!}$

Regresión Poisson (Datos de Conteo)

- Variable objetivo son conteos: $y_i \in \{0, 1, 2, 3, \dots\}$
- Distribución Poisson: $P(y = k) = \frac{\lambda^k e^{-\lambda}}{k!}$

$$\mathcal{L}(\mathbf{w}) = \sum_{i=1}^N \left[e^{\mathbf{w}^T \phi(\mathbf{x}_i)} - y_i \cdot \mathbf{w}^T \phi(\mathbf{x}_i) \right]$$

donde $\lambda = e^{\mathbf{w}^T \phi(\mathbf{x})}$

Regresión Poisson (Datos de Conteo)

- Variable objetivo son conteos: $y_i \in \{0, 1, 2, 3, \dots\}$
- Distribución Poisson: $P(y = k) = \frac{\lambda^k e^{-\lambda}}{k!}$

$$\mathcal{L}(\mathbf{w}) = \sum_{i=1}^N \left[e^{\mathbf{w}^T \phi(\mathbf{x}_i)} - y_i \cdot \mathbf{w}^T \phi(\mathbf{x}_i) \right]$$

donde $\lambda = e^{\mathbf{w}^T \phi(\mathbf{x})}$

```
from sklearn.linear_model import PoissonRegressor
model = PoissonRegressor(alpha=0.0)
model.fit(X, y)
```

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$$

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$$

$$\text{MAPE} = \frac{100\%}{N} \sum_{i=1}^N \left| \frac{y_i - \hat{y}_i}{y_i} \right|$$

¡Cuidado!

No definido cuando $y_i = 0$

Symmetric MAPE (sMAPE)

$$\text{sMAPE} = \frac{100\%}{N} \sum_{i=1}^N \frac{2|y_i - \hat{y}_i|}{|y_i| + |\hat{y}_i|}$$

Symmetric MAPE (sMAPE)

$$\text{sMAPE} = \frac{100\%}{N} \sum_{i=1}^N \frac{2|y_i - \hat{y}_i|}{|y_i| + |\hat{y}_i|}$$

- **Simétrico:** no diferencia sobre/subestimación
- **Acotado:** entre 0% y 200%
- **Robusto:** funciona con $y_i = 0$

Symmetric MAPE (sMAPE)

$$\text{sMAPE} = \frac{100\%}{N} \sum_{i=1}^N \frac{2|y_i - \hat{y}_i|}{|y_i| + |\hat{y}_i|}$$

- **Simétrico:** no diferencia sobre/subestimación
- **Acotado:** entre 0% y 200%
- **Robusto:** funciona con $y_i = 0$

Example

$y_i = 10, \hat{y}_i = 8$:

$$\text{sMAPE} = \frac{2|10 - 8|}{10 + 8} \times 100\% = 22.2\%$$

Coeficiente de Determinación (R^2)

$$R^2 = 1 - \frac{\sum_{i=1}^N (y_i - \hat{y}_i)^2}{\sum_{i=1}^N (y_i - \bar{y})^2}, \quad \bar{y} = \frac{1}{N} \sum_{i=1}^N y_i$$

Coeficiente de Determinación (R^2)

$$R^2 = 1 - \frac{\sum_{i=1}^N (y_i - \hat{y}_i)^2}{\sum_{i=1}^N (y_i - \bar{y})^2}, \quad \bar{y} = \frac{1}{N} \sum_{i=1}^N y_i$$

- $R^2 = 1$: predicción perfecta
- $R^2 = 0$: modelo no mejor que la media
- $R^2 < 0$: modelo peor que la media

Coeficiente de Determinación (R^2)

$$R^2 = 1 - \frac{\sum_{i=1}^N (y_i - \hat{y}_i)^2}{\sum_{i=1}^N (y_i - \bar{y})^2}, \quad \bar{y} = \frac{1}{N} \sum_{i=1}^N y_i$$

- $R^2 = 1$: predicción perfecta
- $R^2 = 0$: modelo no mejor que la media
- $R^2 < 0$: modelo peor que la media

$$R_{\text{adj}}^2 = 1 - \frac{(1 - R^2)(N - 1)}{N - p - 1}$$

donde p = número de características. Penaliza modelos complejos.

Comparación de Métricas

Métrica	Rango	Robusta	Interpretación
MSE	$[0, \infty)$	No	Error cuadrático
MAE	$[0, \infty)$	Sí	Error absoluto
MAPE	$[0, \infty)$	No	Error porcentual
sMAPE	$[0, 200]$	Sí	Error % simétrico
R^2	$(-\infty, 1]$	No	Varianza explicada

Comparación de Métricas

Métrica	Rango	Robusta	Interpretación
MSE	$[0, \infty)$	No	Error cuadrático
MAE	$[0, \infty)$	Sí	Error absoluto
MAPE	$[0, \infty)$	No	Error porcentual
sMAPE	$[0, 200]$	Sí	Error % simétrico
R^2	$(-\infty, 1]$	No	Varianza explicada

- **MAE**: interpretación directa
- **sMAPE**: comparación entre escalas
- R^2 : poder explicativo

Transformación Polinómica

- Una característica x genera:

$$\phi(x) = [1, x, x^2, x^3, \dots, x^d]$$

- Dos características x_1, x_2 (grado 2):

$$\phi(x_1, x_2) = [1, x_1, x_2, x_1^2, x_1x_2, x_2^2]$$

Transformación Polinómica

- Una característica x genera:

$$\phi(x) = [1, x, x^2, x^3, \dots, x^d]$$

- Dos características x_1, x_2 (grado 2):

$$\phi(x_1, x_2) = [1, x_1, x_2, x_1^2, x_1x_2, x_2^2]$$

$$\mathbf{x}_{\text{poly}} = \begin{bmatrix} 1 & x_1 & x_1^2 & \dots & x_1^d \\ 1 & x_2 & x_2^2 & \dots & x_2^d \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_m & x_m^2 & \dots & x_m^d \end{bmatrix}$$

Transformación Polinómica

- Una característica x genera:

$$\phi(x) = [1, x, x^2, x^3, \dots, x^d]$$

- Dos características x_1, x_2 (grado 2):

$$\phi(x_1, x_2) = [1, x_1, x_2, x_1^2, x_1x_2, x_2^2]$$

$$\mathbf{X}_{\text{poly}} = \begin{bmatrix} 1 & x_1 & x_1^2 & \dots & x_1^d \\ 1 & x_2 & x_2^2 & \dots & x_2^d \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_m & x_m^2 & \dots & x_m^d \end{bmatrix}$$

¡Atención!

Número de características: $\binom{n+d}{d}$. Para $n = 10, d = 4 \rightarrow$ ¡1001 características!

PolynomialFeatures

- `degree`: grado máximo (default=2)
- `interaction_only`: solo interacciones
- `include_bias`: incluir intercepto

PolynomialFeatures

- degree: grado máximo (default=2)
- interaction_only: solo interacciones
- include_bias: incluir intercepto

Example

$X = [[0, 1], [2, 3], [4, 5]]$, degree=2:

$$\text{Transformado} = \begin{bmatrix} 1 & 0 & 1 & 0 & 0 & 1 \\ 1 & 2 & 3 & 4 & 6 & 9 \\ 1 & 4 & 5 & 16 & 20 & 25 \end{bmatrix}$$

Pipeline: Polynomial + Linear Regression

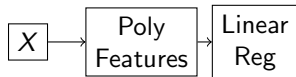
```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression

model = make_pipeline(
    PolynomialFeatures(degree=3),
    LinearRegression()
)
model.fit(X, y)
```

Pipeline: Polynomial + Linear Regression

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression

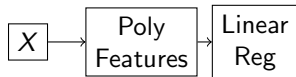
model = make_pipeline(
    PolynomialFeatures(degree=3),
    LinearRegression()
)
model.fit(X, y)
```



Pipeline: Polynomial + Linear Regression

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression

model = make_pipeline(
    PolynomialFeatures(degree=3),
    LinearRegression()
)
model.fit(X, y)
```



- **Ventaja:** transformación + regresión en un solo objeto
- **Alternativa:** SplineTransformer para B-splines

Sensor RTD - Calibración

- Sensor RTD: resistencia varía con temperatura
- Relación $R(T)$ es no lineal
- Modelo polinómico grado 3-5

- Sensor RTD: resistencia varía con temperatura
 - Relación $R(T)$ es no lineal
 - Modelo polinómico grado 3-5
-
- $T \in [0, 100] \text{ } ^\circ\text{C}$
 - $R(T) = R_0(1 + \alpha T + \beta T^2 + \gamma T^3) + \epsilon$
 - $R_0 = 100, \alpha = 3.85 \times 10^{-3}$
 - $\beta = -6.8 \times 10^{-6}, \gamma = 2.1 \times 10^{-9}$

$$T(R) = w_0 + w_1 R + w_2 R^2 + w_3 R^3$$

$$T(R) = w_0 + w_1 R + w_2 R^2 + w_3 R^3$$

```
X = R.reshape(-1, 1)
y = T

model = make_pipeline(
    PolynomialFeatures(degree=3),
    LinearRegression()
)
model.fit(X, y)
```

$$T(R) = w_0 + w_1 R + w_2 R^2 + w_3 R^3$$

```
X = R.reshape(-1, 1)
```

```
y = T
```

```
model = make_pipeline(  
    PolynomialFeatures(degree=3),  
    LinearRegression()  
)  
model.fit(X, y)
```

Recomendación

Usar **Ridge** en lugar de LinearRegression para datos ruidosos.

¿Qué son los Residuos?

$$e_i = y_i - \hat{y}_i$$

¿Qué son los Residuos?

$$e_i = y_i - \hat{y}_i$$

- **Media cero:** $\bar{e} \approx 0$
- **Homocedasticidad:** varianza constante
- **No autocorrelación** (series temporales)
- **Normalidad** (para intervalos de confianza)
- **Aleatorios:** sin patrón visible

- Detecta heterocedasticidad
- Busca patrones sistemáticos

- Detecta heterocedasticidad
- Busca patrones sistemáticos
- Verifica normalidad
- Puntos en diagonal = normal

Interpretación de Patrones

Patrón	Indica	Solución
Embudo	Heterocedasticidad	Transformar y
Curva	Relación no lineal	Añadir términos polinómicos
Outliers	Datos anómalos	Remover o usar regresión robusta
Autocorrelación	Dependencia temporal	Modelos de series temporales

Interpretación de Patrones

Patrón	Indica	Solución
Embudo	Heterocedasticidad	Transformar y
Curva	Relación no lineal	Añadir términos polinómicos
Outliers	Datos anómalos	Remover o usar regresión robusta
Autocorrelación	Dependencia temporal	Modelos de series temporales

```
residuals = y_test - model.predict(X_test)
plt.scatter(y_pred, residuals)
plt.axhline(y=0, color='red', linestyle='--')
```

Dataset Académico

- **Objetivo:** predecir nota final
- **Características:**
 - Horas de estudio semanales
 - Horas de sueño
 - Asistencia a clases
 - Nota del primer parcial
- **Relación no lineal:** rendimientos decrecientes

Dataset Académico

- **Objetivo:** predecir nota final
- **Características:**
 - Horas de estudio semanales
 - Horas de sueño
 - Asistencia a clases
 - Nota del primer parcial
- **Relación no lineal:** rendimientos decrecientes

Horas	Sueño	Asistencia	Nota Final
5	7	80	3.0
10	6	90	3.8
15	5	95	4.2
20	4	98	4.3

Modelo Polinómico

$$\phi(\mathbf{x}) = [1, x_1, x_2, x_3, x_4, x_1^2, x_1x_2, \dots, x_4^2]$$

Total: $\binom{4+2}{2} = 15$ características.

$$\phi(\mathbf{x}) = [1, x_1, x_2, x_3, x_4, x_1^2, x_1x_2, \dots, x_4^2]$$

Total: $\binom{4+2}{2} = 15$ características.

```
model = make_pipeline(  
    StandardScaler(),  
    PolynomialFeatures(degree=2, include_bias=False),  
    Ridge(alpha=1.0)  
)
```

Resultados

Grado	Train R^2	Test R^2	Test MSE	Test MAE	sMAPE
1	0.65	0.60	0.42	0.48	12.5%
2	0.78	0.75	0.28	0.35	9.2%
3	0.85	0.72	0.35	0.40	10.8%
4	0.92	0.58	0.48	0.52	13.5%

Grado	Train R^2	Test R^2	Test MSE	Test MAE	sMAPE
1	0.65	0.60	0.42	0.48	12.5%
2	0.78	0.75	0.28	0.35	9.2%
3	0.85	0.72	0.35	0.40	10.8%
4	0.92	0.58	0.48	0.52	13.5%

Sobreajuste

Grado 4: buen entrenamiento pero pobre en prueba.

Ridge Regression (L2)

$$J(\mathbf{w}) = \frac{1}{2m} \sum_{i=1}^m (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^n w_j^2$$

Ridge Regression (L2)

$$J(\mathbf{w}) = \frac{1}{2m} \sum_{i=1}^m (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^n w_j^2$$

- Penaliza coeficientes grandes
- Reduce varianza
- alpha controla la fuerza

Ridge Regression (L2)

$$J(\mathbf{w}) = \frac{1}{2m} \sum_{i=1}^m (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^n w_j^2$$

- Penaliza coeficientes grandes
- Reduce varianza
- alpha controla la fuerza

```
from sklearn.linear_model import Ridge  
Ridge(alpha=1.0)
```

Lasso Regression (L1)

$$J(\mathbf{w}) = \frac{1}{2m} \sum_{i=1}^m (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^n |w_j|$$

Lasso Regression (L1)

$$J(\mathbf{w}) = \frac{1}{2m} \sum_{i=1}^m (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^n |w_j|$$

- Selección automática de características
- Coeficientes pueden ser cero
- Modelos más interpretables

Lasso Regression (L1)

$$J(\mathbf{w}) = \frac{1}{2m} \sum_{i=1}^m (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^n |w_j|$$

- Selección automática de características
- Coeficientes pueden ser cero
- Modelos más interpretables

```
from sklearn.linear_model import Lasso  
Lasso(alpha=0.1)
```

Lasso Regression (L1)

$$J(\mathbf{w}) = \frac{1}{2m} \sum_{i=1}^m (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^n |w_j|$$

- Selección automática de características
- Coeficientes pueden ser cero
- Modelos más interpretables

```
from sklearn.linear_model import Lasso  
Lasso(alpha=0.1)
```

$$J(\mathbf{w}) = \frac{1}{2m} \sum_{i=1}^m (y_i - \hat{y}_i)^2 + \lambda_1 \sum |w_j| + \lambda_2 \sum w_j^2$$

Selección con Grid Search

```
param_grid = {  
    'polynomialfeatures__degree': [1, 2, 3, 4, 5],  
    'ridge__alpha': [0.1, 1.0, 10.0]  
}  
  
grid = GridSearchCV(pipeline, param_grid, cv=5)  
grid.fit(X_train, y_train)
```

Selección con Grid Search

```
param_grid = {  
    'polynomialfeatures__degree': [1, 2, 3, 4, 5],  
    'ridge__alpha': [0.1, 1.0, 10.0]  
}
```

```
grid = GridSearchCV(pipeline, param_grid, cv=5)  
grid.fit(X_train, y_train)
```

- **Mejor grado:** $d = 2$
- **Mejor alpha:** $\alpha = 1.0$
- **Test score:** $R^2 = 0.76$

Interpretación de Coeficientes

$$y = w_0 + w_1x + w_2x^2$$

- $w_2 > 0$: curvatura positiva
- $w_2 < 0$: curvatura negativa

Interpretación de Coeficientes

$$y = w_0 + w_1x + w_2x^2$$

- $w_2 > 0$: curvatura positiva
- $w_2 < 0$: curvatura negativa

Example

Para horas de estudio:

$$\text{Nota} = 2.1 + 0.15 \cdot \text{horas} - 0.003 \cdot \text{horas}^2$$

Interpretación: cada hora extra es menos efectiva (rendimientos decrecientes).

- 1 La regresión polinómica **transforma características**
- 2 PolynomialFeatures genera combinaciones polinómicas
- 3 **Pipeline**: transformación + regresión
- 4 El sobreajuste se controla con **regularización**
- 5 **Validación cruzada** selecciona el grado óptimo
- 6 **Análisis de residuos** diagnostica el modelo

- **Estandarizar** características
- Usar **Ridge** o **Lasso** para regularización
- **Visualizar** datos para entender la relación
- No usar grados altos con pocos datos
- **Validación cruzada** para evaluar generalización
- Analizar **residuos** para detectar problemas
- Usar **múltiples métricas**

¡A programar!

- 1 Generar datos del sensor RTD
- 2 Ajustar modelos con diferentes grados
- 3 Aplicar al dataset académico
- 4 Analizar residuos
- 5 Comparar métricas (MSE, MAE, MAPE, sMAPE, R^2)
- 6 Seleccionar mejor modelo con Grid Search
- 7 Visualizar e interpretar resultados

Transformaciones de Variables

$$y^{(\lambda)} = \begin{cases} \frac{y^\lambda - 1}{\lambda} & \lambda \neq 0 \\ \log(y) & \lambda = 0 \end{cases}$$

Transformaciones de Variables

$$y^{(\lambda)} = \begin{cases} \frac{y^\lambda - 1}{\lambda} & \lambda \neq 0 \\ \log(y) & \lambda = 0 \end{cases}$$

$$y^{(\lambda)} = \begin{cases} \frac{(y+1)^\lambda - 1}{\lambda} & \lambda \neq 0, y \geq 0 \\ \log(y + 1) & \lambda = 0, y \geq 0 \\ \frac{-(-y+1)^{2-\lambda} - 1}{2-\lambda} & \lambda \neq 2, y < 0 \\ \log(-y + 1) & \lambda = 2, y < 0 \end{cases}$$

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2}$$

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2}$$

$$\text{MASE} = \frac{\frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|}{\frac{1}{N-1} \sum_{i=2}^N |y_i - y_{i-1}|}$$

- Usado en series temporales
- $\text{MASE} < 1$: modelo mejor que naïve

Métrica	Ecuación
MSE	$\frac{1}{N} \sum (y_i - \hat{y}_i)^2$
RMSE	$\sqrt{\frac{1}{N} \sum (y_i - \hat{y}_i)^2}$
MAE	$\frac{1}{N} \sum y_i - \hat{y}_i $
MAPE	$\frac{100}{N} \sum \left \frac{y_i - \hat{y}_i}{y_i} \right $
sMAPE	$\frac{100}{N} \sum \frac{2 y_i - \hat{y}_i }{ y_i + \hat{y}_i }$
R^2	$1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$
R^2_{adj}	$1 - \frac{(1 - R^2)(N - 1)}{N - p - 1}$

¿Preguntas?

`notebook_regresion_polinomica.ipynb`